

en Tiempo Real

Plan de implementación · La Liga 26/27 App iOS

Proyecto	La Liga 26/27 Tracker (iOS 17+, SwiftUI, @Observable)
Bundle ID	com.cornellana.OpencodeTest.-7
Backend	Node.js 20 · Docker · QNAP TBS-h574TX "CORNENAS"
Túnel	Cloudflare Tunnel → laliga-api.cornellanas.net
Fecha	Julio 2026
Autor	cornellana@mac.com

Este documento describe la implementación completa de **avisos push en tiempo real** para la app de seguimiento de La Liga 26/27. El sistema notifica al usuario cuando un partido de sus equipos favoritos registra un gol, penalti, expulsión o inicio/final — **incluso con la app cerrada**. La arquitectura combina APNs de Apple con un backend Node.js desplegado como contenedor Docker en el NAS personal del usuario (QNAP CORNENAS), expuesto al exterior mediante el Cloudflare Tunnel ya existente en la red.

1. Análisis coste / beneficio y alternativas
2. Arquitectura del sistema (diagrama)
3. Parte A — App iOS
4. Parte B — Backend Node.js en el NAS QNAP
5. Archivos críticos
6. Orden de construcción paso a paso
7. Verificación end-to-end
8. Riesgos y notas de seguridad

1. Análisis coste / beneficio

¿Qué se gana?

- Avisos push reales en el bolsillo — **con la app cerrada** — al marcar, penalti, expulsión, inicio/final de los equipos resaltados.
- Integrado con los ajustes ya existentes: equipos resaltados = suscritos; sin apps de terceros.
- Infraestructura 100% propia: NAS personal + Cloudflare Tunnel ya pagados. **Coste incremental = cero.**

Desglose de esfuerzo

Bloque	Esfuerzo	Quién	Detalles
Clave APNs .p8 + portal Apple	15–20 min	Tú (manual)	Developer Portal → Keys → + Anotar Key ID y Team ID
App iOS (A1–A5)	Medio ~3–4 h código	Yo (código)	AppDelegate, entitlements, NotificationService sección Avisos, sync HighlightSettings
Backend Node (B0–B4)	Alto ~5–6 h código	Yo (código)	Poller SofaScore, diff incidentes, APNs HTTP/2 SQLite, Dockerfile, docker-compose
Despliegue NAS + Cloudflare	30–60 min	Tú (consola)	Container Station → imagen Docker Nuevo hostname en el túnel existente
Pruebas end-to-end	Variable	Ambos	Requiere dispositivo físico + partido en vivo

Comparativa de alternativas

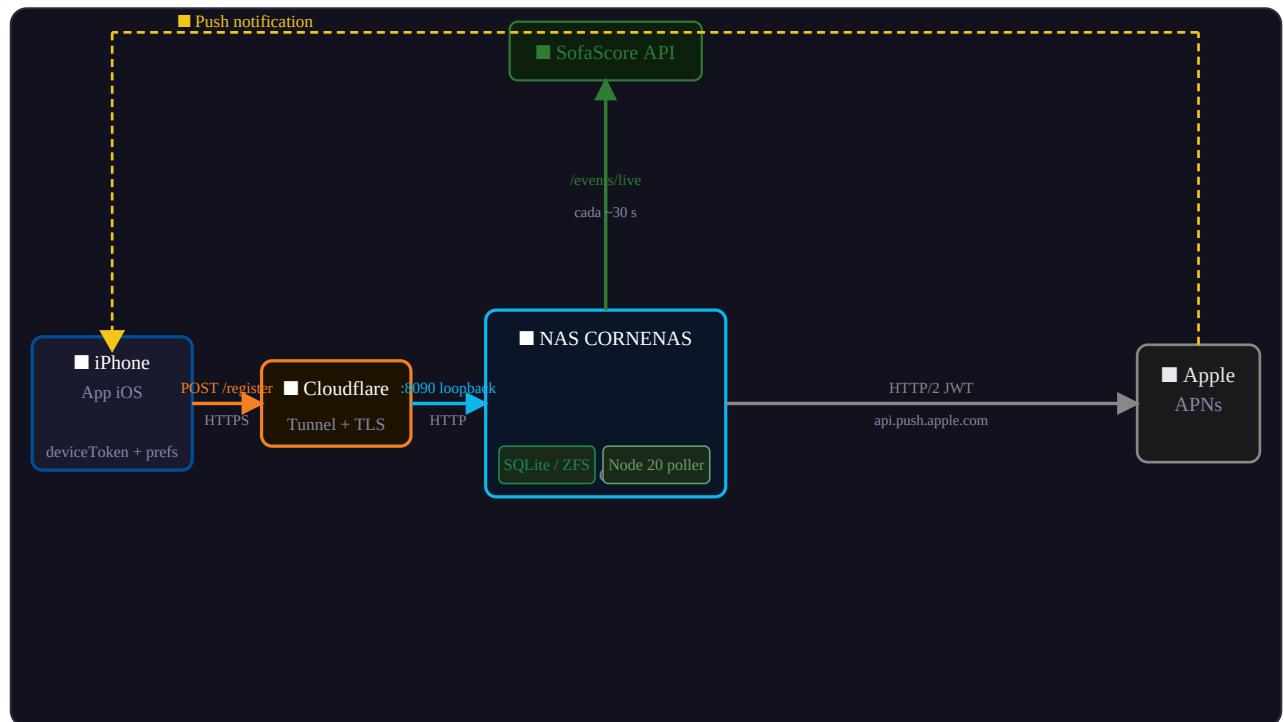
Alternativa	Esfuerzo	Servidor	App cerrada	Mant.
SofaScore/FotMob (apps existentes)	Cero	No aplica	■ Sí	Mínimo
Solo app abierta (polling interno)	Bajo ~2h	No	■ No	Bajo
Este plan completo	Alto ~10h + NAS	Sí (QNAP)	■ Sí	Medio API no oficial

Recomendación

Si el objetivo es **aprender y disfrutar construyendo** el sistema (el NAS ya tiene Docker y Cloudflare Tunnel — la infraestructura no cuesta nada extra), el plan completo tiene todo el sentido y es un proyecto técnicamente interesante. Si el objetivo es **solo recibir el aviso**, SofaScore/FotMob ya lo ofrecen gratis con mantenimiento cero. La opción intermedia (refresco en vivo dentro de la app, sin servidor) cubre el uso más común con el 20% del esfuerzo.

2. Arquitectura del sistema

El sistema opera en dos flujos independientes: el **registro** (iPhone → Cloudflare Tunnel → NAS) y la **notificación** (NAS → SofaScore API → APNs → iPhone).



- Cloudflare Tunnel — ingreso público TLS sin abrir puertos del router
- Docker / NAS CORNENAS — Container Station, volumen ZFS
- SofaScore API (no oficial) — sondeo cada ~30 s, cabecera User-Agent
- Apple APNs — HTTP/2, JWT ES256 firmado con clave .p8
- Línea punteada — trayecto APNs → dispositivo (push notification)

Flujo de registro

- La app iOS obtiene un **deviceToken** de APNs al arrancar (si el usuario dio permiso).
- NotificationService envía `POST https://laliga-api.cornellanas.net/register` con `{deviceToken, environment, teams[], prefs}`.
- Cloudflare Tunnel recibe la petición `HTTPS`, termina el TLS y la reenvía en `HTTP` al puerto 8090 del contenedor.
- El servidor Node hace un *upsert* en la tabla `SQLite subscriptions` por `deviceToken`.

Flujo de notificación (poller ~30 s)

- `GET /api/v1/sport/football/events/live` — filtra `uniqueTournament.id === 8`.
- Cruza equipos en juego con equipos suscritos usando `teamMatches()` (fuzzy, tolerante a tildes).
- Para cada partido: `GET /api/v1/event/{id}/incidents + event.status.type`.
- Diff por `incident.id` contra `seen_incidents (SQLite)`. Solo procesa incidentes nuevos.
- Clasifica: `gol` → ■ + marcador + jugador + `min · tarjeta roja` → ■ `Expulsión · penalti · inicio/final`.
- Selecciona devices con la preferencia activada → envía push APNs `HTTP/2`.

3. Parte A — App iOS

A1. Capabilities y entitlements

- Activar **Push Notifications** en Signing & Capabilities → genera entitlement aps-environment.
- Crear 27/27/27.entitlements con aps-environment = development (debug). Enlazar en project.pbxproj.
- Activar **Background Modes** → **Remote notifications** → UIBackgroundModes = [remote-notification].
- Portal Apple: **Keys** → +, habilitar APNs → anotar Key ID y Team ID. Habilitar Push en App ID.

A2. AppDelegate en _7App.swift

- Añadir @UIApplicationDelegateAdaptor(AppDelegate.self).
- didFinishLaunching: delegado UNUserNotificationCenter · requestAuthorization([.alert,.sound,.badge]) · si concede: registerForRemoteNotifications().
- didRegisterForRemoteNotificationsWithDeviceToken: Data → hex → pasar a NotificationService.
- userNotificationCenter(_:willPresent:): devuelve [.banner, .sound] (avisos en primer plano).

A3. Nuevo NotificationService.swift

- Clase @Observable @MainActor. Guarda deviceToken en UserDefaults.
- func sync(teams:[String], prefs:NotificationPrefs) → POST https://laliga-api.cornellanas.net/register. Mismo idiom URLSession async del proyecto.
- Entorno: debug → "sandbox"; release → "production". Llamado al recibir el token y en cada cambio.

A4. Extender HighlightSettings.swift

- struct NotificationPrefs: Codable, Equatable con enabled, goals, penalties, redCards, startEnd.
- Nueva propiedad var notifications: NotificationPrefs { didSet { persist(); onChange?() } }, clave notification_prefs_v1.
- Callback var onChange: (() -> Void)? → dispara NotificationService.sync(...) al cambiar equipos o prefs.

A5. Sección 'Avisos' en HighlightSettingsSheet.swift

- Nueva Section("Avisos") en el List existente (mismo estilo dark mode, español).
- Toggle maestro **Activar avisos** — si se activa por primera vez, solicita permiso con UNUserNotificationCenter.requestAuthorization.
- Toggles: **Goles** · **Penaltis** · **Expulsiones** · **Inicio y final**.
- Fila de estado de permiso — si está denegado, enlace a Ajustes del sistema.
- Footer: *"Recibirás avisos de los partidos de tus equipos resaltados."*

4. Parte B — Backend Node.js en el NAS QNAP

B0. Alojamiento en CORNENAS (QNAP TBS-h574TX / QuTS hero)

- **Empaquetado:** contenedor Docker en Container Station. Repo con Dockerfile (base node:20-alpine) + docker-compose.yml. Puerto solo loopback: 127.0.0.1:8090:8090.
- **Ingreso público:** nuevo public hostname en el Cloudflare Tunnel ya existente: laliga-api.cornellanas.net → http://localhost:8090. Sin abrir puertos en el router. Inmune a la IP dinámica de Movistar.
- **Persistencia:** volumen montado en share ZFS para el fichero SQLite — sobrevive a reinicios del contenedor y del NAS.
- **Secretos:** APNS_KEY_ID, APNS_TEAM_ID, APNS_BUNDLE_ID, APNS_KEY como variables de entorno del contenedor. El .p8 nunca en git.
- **Auto-start:** restart: always en docker-compose.

B1–B2. Stack y endpoints

- Node 20. Express en puerto 8090. HTTP/2 nativo (node:http2) para APNs. JWT ES256 con jsonwebtoken, cacheado ~50 min.
- SQLite (better-sqlite3) sobre el volumen ZFS.
- POST /register — upsert por deviceToken. POST /unregister (opcional). GET /health → 200.

B3. Bucle de sondeo (~30 s)

- GET /api/v1/sport/football/events/live — filtra uniqueTournament.id === 8.
- Cruza equipos con suscritos usando teamMatches() (fuzzy, portado de SofaScoreService.swift).
- Para cada partido: GET /api/v1/event/{id}/incidents. Diff por incident.id contra SQLite.
- Clasifica: gol / expulsión / penalti / inicio–final. Selecciona devices con pref activada → envía APNs.

B4. Envío APNs (HTTP/2)

- Host: api.sandbox.push.apple.com (sandbox) / api.push.apple.com (producción).
- JWT ES256 con .p8, cacheado ~50 min. Headers: apns-topic: com.cornellana.OpencodeTest.-7.
- Payload: { aps: { alert: { title, body }, sound: "default" } }.
- Respuesta 410 / BadDeviceToken → purgar suscripción de SQLite.

5. Archivos críticos

Archivo	Estado	Cambio
27/27/_7App.swift	Modificar	@UIApplicationDelegateAdaptor + AppDelegate (permiso, token, delegado UN)
27/NotificationService.swift	Nuevo	Guarda deviceToken · POST /register al NAS
27/HighlightSettings.swift	Modificar	NotificationPrefs · clave v2 · hook onChange
27/HighlightSettingsSheet.swift	Modificar	Section 'Avisos' + Toggles maestro + eventos
27/27/27.entitlements	Nuevo	aps-environment · UIBackgroundModes remote-notification
backend/server.js	Nuevo	/register · /health · Express
backend/poller.js	Nuevo	Loop 30s · SofaScore incidents · diff · APNs
backend/apns.js	Nuevo	HTTP/2 + JWT ES256 + envío push
backend/store.js	Nuevo	SQLite better-sqlite3 (suscripciones + vistos)
backend/Dockerfile + docker-compose.yml	Nuevo	node:20-alpine · puerto 8090 loopback · volumen ZFS
Cloudflare Tunnel (NAS)	Config	laliga-api.cornellanas.net → localhost:8090

6. Orden de construcción

1. Portal Apple	Crear .p8 → Key ID + Team ID. Habilitar Push en App ID.
2. Backend local	server.js + store.js + apns.js. Push manual de prueba desde el Mac.
3. Despliegue NAS	Container Station + volumen ZFS + env vars. Nuevo hostname en el túnel. Verificar GET /health desde móvil 4G.
4. iOS push básico	Entitlements + AppDelegate + NotificationService. Verificar prompt, token y POST /register en el NAS.
5. Backend poller	poller.js: loop SofaScore + diff + APNs. Loguear incidentes en vivo.
6. iOS ajustes UI	NotificationPrefs + Section 'Avisos' + sync en cambios.
7. End-to-end	Partido en vivo + aviso al marcar con app cerrada. Dedup + persistencia.

7. Verificación end-to-end

Verificación	Cómo
iOS build	BuildProject en Xcode — compila con entitlements y AppDelegate sin errores.
Permiso push	Al activar 'Avisos' aparece el prompt. Device token visible en consola Xcode.
POST /register	El NAS almacena la llamada. Verificable en: docker logs laliga-api.
GET /health NAS	https://laliga-api.cornellanas.net/health responde 200 desde móvil 4G (fuera de casa).
Persistencia SQLite	Contenedor reiniciado conserva suscripciones y seen_incidents.
Push manual	curl + JWT al token real → notificación recibida con app en background.
End-to-end vivo	Equipo en vivo marca → aviso en ≤60 s con app cerrada.
Dedup	El mismo gol no genera dos notificaciones aunque el poller lo vea dos veces.
Fuzzy match	Athletic ↔ Athletic Club · R. Sociedad ↔ Real Sociedad · Atlético ↔ Atletico Madrid.

8. Riesgos y notas de seguridad

■ Riesgo	API SofaScore no oficial Los endpoints /events/live e /incidents no tienen contrato. Si cambian campos o bloquean el sondeo, los avisos dejan de llegar. Mantener sondeo ~30 s con cabecera User-Agent y validar nombres de campo contra respuesta real antes de fijar el parseo.
■ Nota	Sandbox vs producción APNs App en debug → APNs sandbox. TestFlight/App Store → producción. Enviar al host equivocado da error silencioso. El backend usa el campo "environment" del /register para elegir el host correcto.
■ Nota	Dispositivo físico obligatorio El simulador de iOS no recibe push remoto de forma fiable. Todas las pruebas de push deben hacerse en un iPhone real aprovisionado con la cuenta de pago.
■ Nota	Corte de luz/internet en casa El NAS se apaga → el poller se para. Al volver, seen_incidents en SQLite evita duplicados. Caveat asumible para uso personal. Marcar restart:always en docker-compose.
■ Riesgo	Seguridad del .p8 y endpoint La clave .p8 solo en env vars del contenedor (nunca en git). POST /register debe validar el payload. El ingreso es exclusivamente por Cloudflare Tunnel; sin puertos del router abiertos.
✓ Info	JSON estático independiente Los datos de partidos/clasificación en GitHub son independientes. Los avisos vienen 100% del backend y no dependen de ese JSON.